

A Forward Chaining Expert System for Personalized Programming Language Selection

Andi Sholihin

Department of Informatics
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
6025231080@student.its.ac.id

Shintami Chusnul Hidayati

Department of Informatics
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
shintami@its.ac.id

Abstract—Choosing the right programming language is a critical decision for beginners in computer science because it greatly influences their learning experience and future opportunities. This paper presents an expert system designed to assist novices in selecting a programming language that aligns with their preferences and project requirements. The proposed system uses the forward chaining method, a robust approach to expert system development. It draws on comprehensive developer survey data from Stack Overflow, focusing on the top ten most popular programming languages. Our methodology involved formulating evaluative questions to create a decision-making framework supported by a rule-based table, which served as the foundation for the system's knowledge base. Implemented in Python, the expert system achieved a relevance score of 90%, demonstrating its effectiveness in simplifying the programming language selection process for beginners.

Index Terms—expert system, programming languages, forward chaining, interest determination

I. INTRODUCTION

Selecting the appropriate programming language is a critical decision in software development because it significantly affects a project's performance, maintainability, and scalability [1]. The rapid evolution of technology has led to the proliferation of programming languages, each with its strengths, weaknesses, and ideal use cases [2]. This diversity, coupled with the varying requirements of different projects, poses a considerable challenge, particularly for novices and students who need help identifying the most suitable language for their specific needs [3]. The problem is further complicated by the dynamic nature of programming paradigms, industry standards, and trends that influence language popularity and applicability [4].

An expert system, a computer program that captures the knowledge of human experts, offers a promising solution to this challenge [5]. Such systems are designed to replicate the decision-making abilities of human experts, providing recommendations and solutions to problems that typically require specialized knowledge [6]. Expert systems can be invaluable in programming language selection, offering personalized guidance to users, especially beginners who may lack the experience to make independent decisions.

The selection of an appropriate programming language has been extensively studied in the literature, and various decision-support models have been proposed to guide users in making

informed choices. Among the most prominent methodologies is the Analytical Hierarchy Process (AHP), widely applied to prioritize and evaluate criteria in programming language selection. Tavana *et al.* [7] provided an in-depth analysis of AHP's evolution, highlighting its flexibility in addressing complex decision-making problems across diverse fields, including software engineering. Similarly, Abdelnabi [8] used AHP to create a decision model specifically for novice programmers focusing on data analytics applications, demonstrating its effectiveness in structuring and simplifying decision-making processes. However, while AHP and similar multi-criteria decision-making models, such as the one proposed by Maretto *et al.* [9], offer robust frameworks for decision-making, they often rely on static criteria that may not fully capture the dynamic nature of programming language trends and individual user needs.

In recent years, there has been increasing interest in integrating machine learning and artificial intelligence into programming language selection models. Machine learning offers a distinct advantage by enabling the model to learn from large-scale data, identifying patterns that may not be immediately evident through traditional approaches [10]. Farshidi *et al.* [11] introduced a decision model incorporating a detailed analysis of programming language ecosystems across various industry case studies to provide insights into how language selection impacts software quality and project success. Similarly, Ezenwoye [12] explored the challenges of selecting an introductory programming language in educational contexts, emphasizing the need for adaptable systems that cater to diverse learning environments. These studies underscore the importance of flexibility and adaptability in decision support systems, which aligns with the motivation underlying our proposed expert system.

In addition, Jain *et al.* [13] examined the environmental impact of programming language selection and introduced a novel perspective on how language choice affects CO2 emissions—a consideration that may become increasingly relevant as sustainability becomes a key concern in software development. The importance of context-specific factors was further highlighted by Katzy *et al.* [14] and Li *et al.* [15], who investigated the implications of language selection in machine learning models and multi-language software projects,

respectively. These studies suggested that a one-size-fits-all approach is insufficient for programming language selection, thus reinforcing the need for personalized, context-aware recommendations.

Our proposed expert system distinguishes itself from existing approaches by leveraging forward chaining to provide personalized, dynamic recommendations. Unlike AHP-based models, which depend on predefined criteria, the proposed system can adapt to the user's evolving needs and the changing landscape of programming languages. The forward chaining method, as discussed by İmamoğlu and Çetinkaya in the context of rule-based decision support systems [16], allows for a more interactive and iterative decision-making process in which the system gathers information progressively and refines its recommendations based on user input.

Furthermore, the proposed system is designed to be accessible to beginners, addressing the challenges highlighted by Ahmed *et al.* [17] and Shrestha *et al.* [18] regarding the readability, writability, and reliability of programming languages and the difficulties developers face when learning new languages. By focusing on user-specific preferences and project requirements, our system simplifies the decision-making process and enhances the learning experience, providing a tailored pathway for novices to navigate the diverse landscape of programming languages.

In summary, while existing studies have laid a strong foundation for programming language selection through various decision support models and contextual analyses, our proposed expert system offers a novel approach that combines the adaptability of forward chaining with a user-centric design. This enables more personalized and context-sensitive recommendations, making it a valuable tool for beginners and experienced developers in rapidly evolving software development.

The remainder of this paper is organized as follows. Section II describes the methodology employed in developing the expert system, including the formulation of evaluative questions, construction of the knowledge base, and implementation of the forward chaining inference mechanism. Section III presents the experimental setup and results, including an analysis of user feedback collected from a survey. Finally, Section IV concludes the paper by summarizing the key contributions of this study.

II. METHODOLOGY

This section outlines the proposed systematic approach, identifying the most popular programming languages through developer surveys and formulating selection rules for programming languages.

A. Identification of Popular Programming Languages

To enhance the credibility and relevance of this study, we used an annual developer survey conducted by Stack Overflow¹, an online platform for programmers worldwide. This survey was the primary source to determine the popular

¹<https://survey.stackoverflow.co/2023>

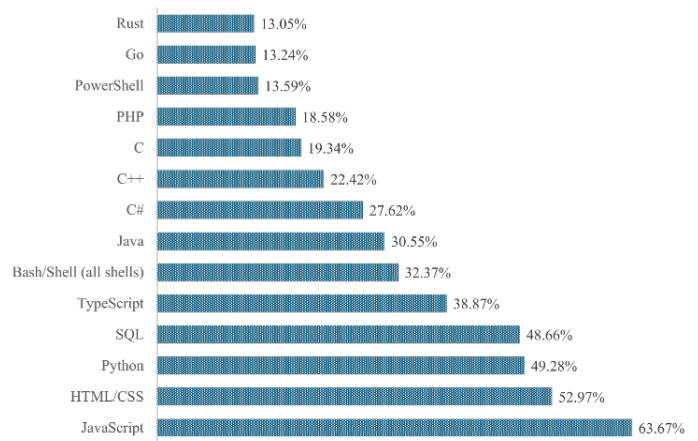


Fig. 1. The identified most popular programming languages.

programming languages in the current industry landscape, as summarized in Fig. 1. Through a detailed analysis of the survey data, we identified prevailing trends and preferences among developers regarding their choice of programming languages. The insights gained from this analysis form a foundation for the subsequent development of programming language selection rules, thus bridging empirical data with practical application in our expert system.

B. Formulation of Programming Language Selection Rules

Establishing robust criteria for selecting programming languages is critical in developing our expert system, which leverages the forward chaining method. The foundational step in this process involved creating evaluative questions designed to capture user requirements and preferences in programming language selection. These questions were crafted to align with key decision-making factors, ensuring their relevance and applicability.

Based on analyzing Stack Overflow survey data, we identified the top ten programming languages that reflect current industry trends. These languages include JavaScript, Python, TypeScript, Java, C#, C++, C, PHP, Go, and Rust. Table I presents a set of questions outlining the suitability of these languages for various software development tasks and programming paradigms.

C. Implementation of Forward Chaining

Creating a forward chaining inference system based on the provided table begins by defining a knowledge base comprising initial facts and a set of rules. The facts represent user responses to the evaluative questions, such as whether users wanted to develop a web frontend or work on data science projects. Each rule in the knowledge base corresponds to a row in the table and is structured as an “if-then” statement: if a user's response satisfies a condition (e.g., “Yes” to developing a web frontend), then the programming languages deemed suitable for that task (e.g., JavaScript, TypeScript) are identified. Fig. 2 outlines the steps involved in the inference

TABLE I
PROGRAMMING LANGUAGE SELECTION RULES

ID	Question	JavaScript	Python	TypeScript	Java	C#	C++	C	PHP	Go	Rust
Q01	Do you want to develop a web frontend?	✓	×	✓	×	×	×	×	×	×	×
Q02	Do you want to develop a web backend?	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Q03	Do you want to develop Desktop Applications?	✓	✓	×	✓	✓	✓	✓	×	✓	×
Q04	Do you want to develop Mobile Applications?	✓	✓	✓	✓	✓	✓	×	×	✓	×
Q05	Do you want to develop Games?	✓	✓	✓	✓	✓	✓	×	×	✓	×
Q06	Do you want to develop AI?	✓	✓	×	✓	✓	✓	×	×	✓	×
Q07	Do you want to develop Data Science?	✓	✓	×	✓	✓	✓	×	×	✓	×
Q08	Do you want to develop Big Data?	✓	✓	×	✓	✓	✓	×	×	✓	×
Q09	Do you want to use an application that supports Object Oriented Programming?	✓	✓	✓	✓	✓	✓	×	✓	✓	✓
Q10	Do you want to use an interpreter-based programming language?	✓	✓	×	×	×	×	×	✓	×	×
Q11	Do you want to use a Compiler based programming language?	×	×	✓	✓	✓	✓	✓	×	✓	✓

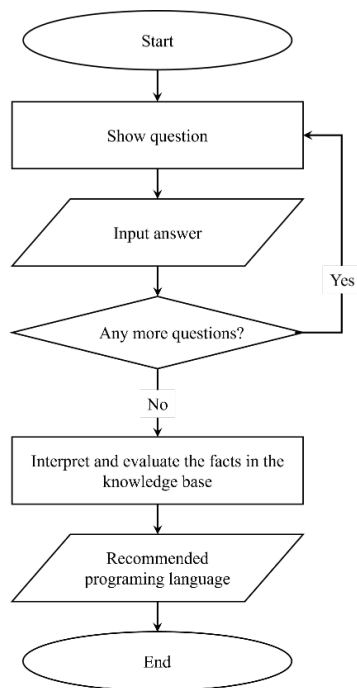


Fig. 2. Flowchart of the inference procedure.

```

Do you want to develop Web Frontend?
Your answer (yes/no): yes
Do you want to develop Web Backend?
Your answer (yes/no): no
Do you want to develop Desktop Applications?
Your answer (yes/no): yes
Do you want to develop Mobile Applications?
Your answer (yes/no): yes
Do you want to develop Games?
Your answer (yes/no): yes
Do you want to develop AI?
Your answer (yes/no): yes
Do you want to develop Data Science?
Your answer (yes/no): yes
Do you want to develop Big Data?
Your answer (yes/no): yes
Do you want to use an application that supports Object Oriented Programming?
Your answer (yes/no): yes
Do you want to use an Interpreter based programming language?
Your answer (yes/no): yes
Do you want to use a Compiler based programming language?
Your answer (yes/no): no
The recommended Programming Language for you is : JavaScript
  
```

Fig. 3. A sample of the inference using an “if-then” rule.

procedure and Fig. 3 provides a sample inference using an “if-then” rule.

The inference engine begins by collecting user input through a series of questions, each designed to capture user requirements and preferences for programming languages. As the user answers each question, these responses are stored as facts in the system. The forward chaining process begins by iteratively evaluating each rule in the knowledge base against the accumulated facts. When a match is found—meaning the user’s facts satisfy a particular rule—the rule is activated, and the corresponding programming languages are added to the list of recommendations.

As the system processes all relevant rules, it compiles a comprehensive list of programming languages that align

with user requirements. If multiple languages are suitable, the system may present all options or prioritize them based on additional criteria, such as popularity or ease of learning. The final recommendations are then presented to the user, who can select the language that best suits their project goals. This process not only aids in making informed decisions but also provides an interactive experience where users can refine their choices according to their evolving preferences.

III. RESULTS AND DISCUSSION

We conducted a series of experiments involving student participation to assess the effectiveness of the developed expert system for programming language selection. The initial step involved inviting students to interact with the proposed system, which allowed them to answer several questions and receive recommendations for programming languages that best suited their needs. After using the system, we gathered feedback from 20 students through a survey designed to evaluate the relevance of the recommendations provided by the system.

The survey results, which are shown in Fig. 4, offer insights into the system’s performance based on direct user feedback. The majority of respondents responded positively to the system’s usefulness, with 14 out of 20 students (70%) finding

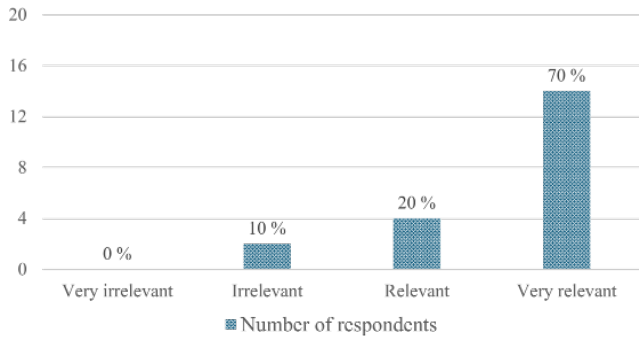


Fig. 4. Relevance scores obtained from the survey.

the proposed system “very relevant” in helping them choose a programming language that matched their needs. Additionally, four students (20%) considered the system “relevant”, while only two students (10%) felt that the system was “not relevant” for programming language selection. Notably, none of the respondents rated the system as “very irrelevant”.

The overwhelming positive response suggests that the expert system is generally well received and is considered a valuable tool to support decision-making in programming language selection. The lack of any ratings for “very irrelevant” further underscores the system’s perceived effectiveness among participants. Therefore, it is understood that the proposed system can be effectively utilized as a supportive resource to help users make informed choices about programming languages.

To quantify the relevance of the system based on user ratings (very relevant = 4, relevant = 3, not relevant = 2, very irrelevant = 1), we calculated the overall percentage of the relevance score $\mathfrak{R}$ using the following formula:

$$\mathfrak{R} = \frac{\sum_{i=1}^{n_r} s_i}{n_r \times \max_i(s_i)} \times 100\%, \quad (1)$$

where n_r is the number of respondents and s_i is the relevance score given by participant i . Given that the maximum relevance value $\max_i(s_i)$ is 4 (very relevant) and from the survey that 14 respondents indicated “very relevant,” 4 indicated “relevant,” 2 indicated “not relevant,” and none indicated “very irrelevant”, our relevance score $\mathfrak{R}$ is 90%.

Therefore, the survey results demonstrate that the programming language selection expert system is highly relevant. This high percentage indicates strong user confidence in the system’s effectiveness in assisting with the programming language selection.

Despite the promising results, the current proposed system has some limitations that should be addressed in future research. Firstly, the system’s effectiveness was evaluated using a relatively small sample size of 20 students, which may limit the generalizability of the findings. Future studies should involve a larger and more diverse group of participants, including professionals and educators, to obtain a more comprehensive evaluation of the system’s relevance and utility across different user demographics. Secondly, the system relies on a predefined set of questions and rules, which may not

capture the full range of programming language preferences and requirements across various domains. Incorporating machine learning techniques could allow the system to adapt and improve over time based on user feedback and evolving trends in programming language usage. Finally, the system currently evaluates programming languages based on general criteria, without accounting for specific project types or industry standards. Future research could explore the integration of domain-specific factors [19], [20], such as industry-specific language requirements, performance benchmarks, and long-term maintainability considerations, to provide more nuanced recommendations.

IV. CONCLUSIONS

This study introduced an expert system for programming language selection, employing forward chaining to provide personalized recommendations. Aimed at addressing the challenges faced by beginners in choosing the most suitable programming language, the system integrates a dynamic knowledge base that adapts to evolving industry trends and user needs. Our evaluation revealed that the system is highly effective, achieving a 90% relevance score. The majority of users found the system valuable in assisting their programming language decisions, underscoring its potential as a practical tool for both novice and experienced developers.

ACKNOWLEDGMENT

This work was partially supported by Institut Teknologi Sepuluh Nopember under Grant No. 1409/PKS/ITS/2023.

REFERENCES

- [1] D. Spinellis, “Choosing a programming language,” *IEEE Software*, vol. 23, no. 4, pp. 62–63, 2006.
- [2] M. Noone and A. Mooney, “Visual and textual programming languages: a systematic review of the literature,” *Journal of Computers in Education*, vol. 5, no. 2, pp. 149–174, 2018.
- [3] C. Kelleher and R. Pausch, “Lowering the barriers to programming: A taxonomy of programming environments and languages for novice programmers,” *ACM Computing Surveys*, vol. 37, no. 2, pp. 83–137, 2005.
- [4] P. Perera, G. Tennakoon, S. Ahangama, R. Panditharathna, and B. Chathuranga, “A systematic mapping of introductory programming languages for novice learners,” *IEEE Access*, vol. 9, pp. 88 121–88 136, 2021.
- [5] B. Reddy and R. Fields, “From past to present: a comprehensive technical review of rule-based expert systems from 1980 – 2021,” in *Proc. ACM Southeast Conference*, 2022, p. 167–172.
- [6] S. C. Hidayati and Y. Anistiyasari, “Body shape calculator: Understanding the type of body shapes from anthropometric measurements,” in *Proc. ACM International Conference on Multimedia Retrieval*, 2021, pp. 461–465.
- [7] M. Tavana, M. Soltanifar, and F. J. Santos-Arteaga, “Analytical hierarchy process: revolution and evolution,” *Annals of Operations Research*, vol. 326, pp. 879–907, 2023.
- [8] A. A. Bany Abdelnabi, “An analytical hierarchical process model to select programming language for novice programmers for data analytics applications,” in *International Arab Conference on Information Technology*, 2019, pp. 128–132.
- [9] L. Maretto, M. Faccio, and D. Battini, “A multi-criteria decision-making model based on fuzzy logic and ahp for the selection of digital technologies,” *IFAC-PapersOnLine*, vol. 55, no. 2, pp. 319–324, 2022.
- [10] Y.-H. Hsieh, S. C. Hidayati, W.-H. Cheng, M.-C. Hu, and K.-L. Hua, “Who’s the best charades player? mining iconic movement of semantic concepts,” in *Proc. International Conference on Multimedia Modeling*, 2014, pp. 231–241.

- [11] S. Farshidi, S. Jansen, and M. Deldar, "A decision model for programming language ecosystem selection: Seven industry case studies," *Inf. Softw. Technol.*, vol. 139, 2021.
- [12] O. Ezenwoye, "What language? - the choice of an introductory programming language," in *Proc. IEEE Frontiers in Education Conference*, 2018, pp. 1–8.
- [13] D. Jain, R. K. Shukla, M. S. Tomar, and P. Sharma, "A study of the impact of programming language selection on co2 emission – a green it initiative," in *Proc. International Conference on Data, Engineering and Applications*, 2020, pp. 1–5.
- [14] J. Katzy, M. Izadi, and A. v. Deursen, "On the impact of language selection for training and evaluating programming language models," in *Proc. IEEE International Working Conference on Source Code Analysis and Manipulation*, 2023, pp. 271–276.
- [15] W. Li, N. Meng, L. Li, and H. Cai, "Understanding language selection in multi-language software projects on github," in *Proc. IEEE/ACM International Conference on Software Engineering: Companion Proceedings*, 2021, pp. 256–257.
- [16] M. Y. İmamoglu and D. Çetinkaya, "A rule based decision support system for programming language selection," in *Proc. International Conference on Knowledge Engineering and Applications*, 2017, pp. 71–75.
- [17] Z. Ahmed, F. J. Kinjol, and I. J. Ananya, "Comparative analysis of six programming languages based on readability, writability, and reliability," in *Proc. International Conference on Computer and Information Technology*, 2021, pp. 1–6.
- [18] N. Shrestha, C. Botta, T. Barik, and C. Parnin, "Here we go again: why is it difficult for developers to learn another programming language?" in *Proc. ACM/IEEE International Conference on Software Engineering*, 2020, pp. 691–701.
- [19] D. D. Thang, S. C. Hidayati, Y.-Y. Chen, W.-H. Cheng, S.-W. Sun, and K.-L. Hua, "A spatial-pyramid scene categorization algorithm based on locality-aware sparse coding," in *Proc. IEEE International Conference on Multimedia Big Data*, 2016, pp. 342–345.
- [20] T. Takayanagi and K. Izumi, "Oincorporating domain-specific traits into personality-aware recommendations for financial applications," *New Generation Computing*, 2024.